Learning: the score and the scorecard

Trunk section: §6.7 Learning.

Talks to: Registers (§6.6), which supply R4, the only training data; the Card (§6.3), which shows the score; Tools and screeners (§6.2), whose screeners the scorecard judges; Platform (§6.10), which schedules training.

Built on: IDEAS parts 1 and 3 (what the score and the scorecard are for, their rules, the open questions); SCORING-TODAY (today's score and scorecard, step by step, and why neither should be copied); METHODOLOGY (the measure, the benchmark, the mirrors, reading the scorecard). The code: `src/scoring/processor.py`, `scorer.py`, `server.py`, `src/sideE/score.js`, `src/training/trainingData.js`, `src/analysis/screenerReport.js`.

Contents

1. The two questions
2. The rules both keep
3. The legacy score
4. The legacy scorecard
5. What failed
6. The new score
7. The new scorecard
8. Contracts
9. How the rebuild is proven

---

1. The two questions

	The score	The scorecard
Asks	of the stocks this tool found before that looked like this one, how many made a good move?	keep this screener, watch it, or drop it?
About	one card, at discovery	one screener, over all its days
Used for	ranking today's cards; filling the shortlist automatically. **Never a filter**	the order of work: collect, cut, then setups
Learns from	the tool's own R4	the tool's own R4

2. The rules both keep

From IDEAS, each learned on the live desk:

1. **Each tool learns only from itself.** Tools are separate experiments.
2. **No look-ahead.** Inputs are what was known at the photo; outcomes only after the entry; ATR from days before.
3. **One good-move line**: `≥ 1.3 ATR` in the trade's direction, from the entry, before the close. A big move is `≥ 2`. The score, the scorecard and the shortlist never grade on different curves.
4. **Entries A and B are never mixed.**
5. **Honest about thin data.** Every number carries its sample, and below a minimum there is no number.
6. **Explainable.** A score says why; a verdict shows its sample and its thresholds.
7. **Unit-free features first**: relations (above or below a level, distance in %, position in a range, the MA stack, PM/ADR) compare a $2 stock with a $200 one; raw prices do not.
8. **The score is fixed at discovery**; re-scored only when the trader changes the bias.
9. **Retrained every day** after the outcomes are in.
10. **The market matters**: prefer what happened in the same regime; fall back to all regimes and say so.

3. The legacy score

The moving parts: training rows in `r4a_train` / `r4b_train`, exported as CSV; a trainer (`processor.py`) writing tables into `src/scoring/outputs-tN/`; a live scorer (`scorer.py`, Flask, `scorer-Tn` on the tool's port + 1); the caller (`sideE/score.js`) at discovery; retraining at 16:20.

Six bases:

Base	Rows	Learns
B1	R4A (entry A)	upR_A
B2	R4A	downR_A
B3	R4A	the larger
B4	R4B (entry B)	upR_B
B5	R4B	downR_B
B6	R4B	the larger

Each base has a **main** table from all rows and a **sub** table per regime.

Building a table:

1. About 60 features: one-hot categoricals (sector, industry, regime, sector bias, themes, catalyst, screener keys, the three trend stages, bias, the unit-free relations) and standardised numerics (price, change, gap, VWAP, the MAs, RVOL, ATR, ADR %, day and month high/low, range positions, cap, float, short %, PM range and PM/ADR, sector score, minutes from the open, the % distances).
2. **PCA**, keeping factors with eigenvalue above 1.
3. Each factor cut into **5** equal-count buckets.
4. Per bucket: mean R and win rate (R **> 1.3**), each normalised by the best bucket of that factor; `raw = (0.5 × mean_norm + 0.5 × winrate_norm) × 100`; `confidence = n ÷ (n + 5)`; **FinalScore = raw × confidence**.
5. Saved: factor importances, one bucket table per factor, the encoder, scaler, PCA and bucket edges. Tables not rebuilt are deleted.

Scoring a card: the base from the bias and entry time (long → upR, short → downR, none → the larger); the regime sub table if it has at least 150 rows (`regimeSampleThreshold`), else main; encode and project; look up each factor's bucket; **score = the plain average** of the bucket scores. The breakdown shows the base, table, sample, confidence and each factor's bucket.

---

4. The legacy scorecard

`/api/analysis/screener-report?entry=A|B`:

1. Read every training row of the chosen entry (captured and uploaded).
2. Credit each row to **every** screener in its `screenerKeys`.
3. Per screener: cards, days, good % (`upR ≥ 1.3`), big % (`upR ≥ 2`), median and average R (of upR), average −R (of downR), **days hit** (share of days whose best card reached 1.3).
4. **Verdict:** under 20 cards or 5 days → not enough data; **keep** when good ≥ 35 % and days hit ≥ 50 %; **watch** when good ≥ 25 % or days hit ≥ 40 %; **drop** otherwise.
5. **Baseline:** all of the tool's rows pooled.

The comparison page (`/screeners/compare`): every screener against the benchmark (it must beat Stocks in Play by 5 points), and each screener against its mirror (**directional**, **both sides work**, **neither beats the plain list**).

---

5. What failed

ID	Fault	Effect	Prevented by
D-D1	the scorer is off on every tool	no card has a score; the auto-rule adds nothing	the model is applied in-process; no scorer process
—	~60 features against a few hundred rows per tool, far fewer per regime	the factors describe noise	a small feature set chosen per tool, validated walk-forward (§6.2)
—	factors averaged equally	the score is pulled to the middle	one fitted model, not an average of buckets
—	buckets normalised by the best bucket	in a table where nothing moves, the best of the bad still scores ~100	absolute, calibrated probabilities
—	the model's win is `> 1.3`, the scorecard's `≥ 1.3`	two curves	one definition (§2.3)
—	PCA factors cannot be explained to a trader	the breakdown says nothing	contributions per named feature, and similar past cards
—	no minimum sample	thin scores look as sure as thick ones	a minimum, and a model published only when it beats the base rate
D-S12	every screener graded on the move up, including bearish ones	short setups read as failures; the mirror comparison is backwards	direction-aware grading
—	the pooled baseline counts a card once per screener that found it	the baseline leans to cards several screeners agree on	each card counted once in the baseline
—	uploaded external rows mixed with captured ones	a verdict can rest on data the tool never collected	uploads excluded unless included on purpose
—	thresholds fixed in code	shown, not tunable	settings, versioned
D-S14	no benchmark while T9 is stopped	no comparison	the benchmark always runs (TOOLS-AND-SCREENERS.md §8.1)
—	dead per-setup grading routes left in `server.py`	dead code beside live	not carried over

6. The new score

6.1 What it predicts

For each tool and each of **six questions** (entry A or B × long, short or either), the **probability that the card makes a good move** (`≥ 1.3 ATR` in that direction from the entry). The card shows the question its bias points to (manual, then automatic) at the tool's chosen entry, as a number 0–100 = the calibrated probability × 100, beside the tool's **base rate** for that question.

6.2 Features

Chosen per tool from a fixed menu, in this order of preference:

1. **unit-free relations**: above or below each level, % distance, range positions, MA stack, PM/ADR band, gap %, ADR %, RVOL;
2. **context**: regime group (up, sideways, down), sector bias, hot sector, catalyst tier and sentiment, minutes from the open at discovery, shortlisted elsewhere;
3. **the tool's own screener keys**, only those with enough cards;
4. **CANSLIM letters**, once enough photographed days carry them.

A feature enters only if it improves the walk-forward score (§6.3).

6.3 The model and its validation

Part	Choice	Why
model	regularised logistic regression per question	small samples; coefficients explain themselves; calibrated by construction
similar cards	the 20 nearest past cards on the standardised features, with how many made a good move	the explanation a trader trusts
regime	regime group as a feature; a separate model per regime group only when it has enough rows	the legacy sub tables rested on a handful of days
validation	**walk-forward by day**: train on days before, test on the next day, over all history	no look-ahead in the evaluation either
publish rule	a new model version is published only if its walk-forward Brier score beats the base rate's	a model no better than "always say the base rate" must not pretend to be
minimum	at least 100 photographed cards over 10 days for the question; else no score, and the card says why	thin data shows no number

Trained by the research worker after the outcomes (16:20), from the tool's R4 only. A model is a **versioned record**: question, features, coefficients, scaler, training window, sample, walk-forward metrics, published or not, and why.

6.4 Applied whenever the card changes

The card engine scores each tool card from the tool's current published model, in-process (a dot product and a logistic), **whenever the card changes**: at discovery, at each re-quote, on a new market state and when the bias changes (today's desk also re-scores on every scan). The score is stored on the tool card with the model version, the sample and a confidence interval. The photo freezes the score of the card it froze, so the model is always judged on the features it saw.

6.5 The breakdown

Tapping the score shows: the question (e.g. "entry B, long"), the probability and the base rate, the sample and its interval, the model version and its walk-forward accuracy, the **features that pushed it up or down** (named, with their values), and **the 20 most similar past cards** with their outcomes.

6.6 The shortlist auto-rule

One minute before the photo: the top N cards (setting, default 5) whose score is at least the setting (default 70) **and** whose model is published. With no published model, nothing is added, and the tool says why.

---

7. The new scorecard

7.1 Direction-aware

Each screener is graded on **its own direction**: a bullish screener on upR, a bearish one (most mirrors) on downR, a direction-agnostic one (T7's gap, the benchmark) on the larger. The direction is a field of the screener version.

7.2 The numbers

Per screener version and entry, from captured rows only:

Number	Formula
cards, days	rows, distinct days
good %	share with R ≥ 1.3 in its direction, with a Wilson 90 % interval
big %	share with R ≥ 2
median and average R	in its direction
average adverse R	the other direction
days hit	share of days whose best card reached 1.3: **the deciding number**

Rows of an older version are shown apart, never pooled with the current one.

7.3 The verdict

Verdict	Rule (settings, versioned; defaults from legacy)
not enough data	under 20 cards or 5 days
keep	good % ≥ 35 **and** days hit ≥ 50 %
watch	good % ≥ 25 **or** days hit ≥ 40 %
drop	otherwise

The thresholds and the sample are printed beside every verdict.

7.4 The two comparisons

1. **Against the benchmark** (the tool’s own baseline, and Stocks in Play): beat it by 5 points of good %, with the intervals shown, so a 3-point gap on a thin sample is never read as proof. The universe caveat is stated: part of a gap can be a difference of universe.
2. **Against its mirror**: same window, same universe, so the difference is direction. **directional**, **both sides work**, or **neither beats the benchmark**.

The tool’s baseline counts **each card once**, whatever the number of screeners that found it.

---

8. Contracts

Owns

Table	Key
`models`	tool, question, version
`scorecards`	tool, screener version, entry, day computed

Reads

From	What
registers	R4 per tool (and uploads only when asked)
tools and screeners	screener versions, direction, mirrors, the benchmark

Read by

Domain	What
card	the tool’s current published model per question
registers	the auto-rule’s scores

Routes (desk-web)

Route	Returns
`GET /api/tools/:tool/models`	model versions with their metrics and publish decision
`GET /api/tools/:tool/scorecard?entry=A|B`	every screener’s numbers and verdict
`GET /api/compare?entry=A|B`	every screener against the benchmark and its mirror

---

9. How the rebuild is proven

1. **Same measure.** The new scorecard on last month’s data reproduces the legacy numbers for bullish screeners exactly, and differs only where direction-aware grading and counting each card once explain it.

2. **Walk-forward report.** For each tool and question: the model’s Brier score against the base rate, by month. A model that does not beat it is not published.
3. **No look-ahead test.** A fixture whose outcome is planted in a feature recorded after the photo must not be usable by the model.
4. **Explanation test.** For a sample of cards, the named contributions sum to the score’s log-odds.